

E-COMMERCE SALES & CUSTOMER ANALYTICS

SQL Portfolio Project | MySQL

Prepared by Tisha Jain
End-to-End Business Analytics Case Study

Database Design • SQL Analysis • Business Insights • Dashboard Plan

1. Executive Summary

I built this end-to-end e-commerce analytics project in MySQL to demonstrate how I translate raw transactional data into business insights. I designed a six-table relational database, created and loaded the dataset, defined business questions, wrote analytical SQL, validated the logic, and interpreted the outputs from a business perspective. My analysis covers revenue, customers, products, categories, cities, order status, payments, inventory risk, segmentation and month-over-month performance.

2. Business Problem Statement

I approached this project as an analyst supporting an e-commerce company. The business stores customer, product, order and payment information across multiple relational tables, but the raw records do not directly answer management questions. My objective was to create a structured SQL analysis that explains sales performance, customer value, product demand and operational risks.

3. Business Objective

My objective was to analyze the company's e-commerce data using MySQL and identify meaningful trends and performance indicators across customers, products, categories, cities, orders and payments. I designed the analysis so that the final outputs can support a management dashboard and can also be defended clearly in a data-analyst interview.

3.1 Questions I set out to answer

- How much valid sales revenue is generated and how does it change by month?
- Which categories and products contribute most to revenue?
- Who are the highest-value and repeat customers?
- Which cities generate the most revenue?
- What is the order-status mix and cancellation rate?
- Which payment methods account for the largest paid amount?
- Which products combine active demand with low stock?
- How concentrated is revenue across customer segments and categories?

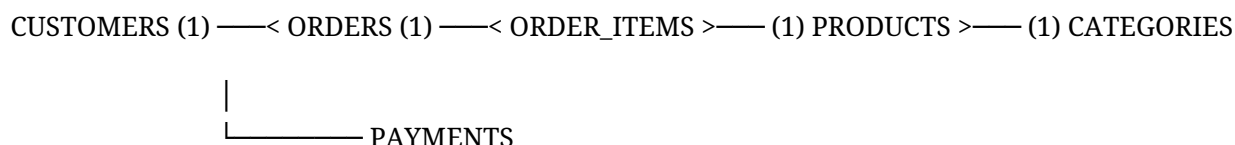
4. Database Architecture

Table	Purpose	Primary Key	Foreign Key(s)
customers	Customer master data	customer_id	—
categories	Category master data	category_id	—
products	Product, category, price and stock	product_id	category_id
orders	Order header, date and status	order_id	customer_id
order_items	Products and quantities within orders	order_item_id	order_id, product_id
payments	Payment date, method, amount and status	payment_id	order_id

4.1 Entity relationships

- One customer can place many orders.
- One category can contain many products.
- One order can contain many order items.
- One product can appear in many order items.
- Each payment record belongs to an order.
- `order_items` acts as the transaction bridge between orders and products.

4.2 ER Diagram



The diagram above represents the logical relationship structure used throughout my SQL analysis.

5. Dataset Overview

Table	Rows
customers	10
categories	5
products	12
orders	15
order_items	28
payments	15

The transactional period covers May and June 2025. The dataset contains 15 orders: 13 non-cancelled orders and 2 cancelled orders. I intentionally used a compact dataset so every output could be checked manually while still demonstrating multi-table analytics and advanced SQL patterns.

6. Analytical Definitions & Assumptions

- For sales analysis, I define a valid order as any order whose status is not 'Cancelled'. This includes Delivered, Shipped and Processing orders in this project.
- Sales revenue = product price $\times$ quantity for non-cancelled orders.
- Payment-method analysis is separate and uses only rows where payment_status = 'Paid'.
- Average order value = valid sales revenue $\div$ distinct valid orders.
- The customer-segment thresholds are project-defined analytical thresholds: High Value $\geq$ ₹7,000; Medium Value $\geq$ ₹3,000; otherwise Low Value.
- Product revenue bands are project-defined: High Revenue $\geq$ ₹5,000; Medium Revenue $\geq$ ₹2,000; Low Revenue $<$ ₹2,000; No Sales = ₹0.
- Because the dataset contains only two months, month-over-month movement is treated as a short-period signal, not a long-term forecast.

7. SQL Skills Demonstrated

Technique	Application
INNER JOIN	Combined related transactional/master tables
LEFT JOIN	Preserved customers/products with no qualifying sales
GROUP BY / HAVING	Built business summaries and post-aggregation filters
CASE	Conditional metrics and segmentation
COALESCE	Converted missing sales values to zero
CTEs	Separated complex analytical logic into readable stages
Subqueries	Compared values against averages and totals
RANK / DENSE_RANK	Created business rankings while handling ties
ROW_NUMBER	Selected Top-N rows within groups
LAG	Compared current and previous month revenue
Window SUM	Calculated cumulative revenue and contribution
DATE_FORMAT	Built monthly reporting periods

8. Verified Executive Results

KPI	Verified Result
Total sales revenue	₹49,765
Total orders	15
Valid / non-cancelled orders	13
Cancelled orders	2
Cancellation rate	13.33%
Average order value	₹3,828.08
Total quantity sold	35 units
May 2025 revenue	₹32,877
June 2025 revenue	₹16,888
June MoM change	-48.63%
Top category	Electronics — ₹17,492
Top revenue product	Smart Watch — ₹8,997
Top quantity product	Sunscreen — 5 units
Top customer	Aarav Sharma — ₹11,192
Top city	Delhi — ₹18,387

8.1 Category performance

Category	Revenue	Contribution
Electronics	₹17,492	35.15%
Clothing	₹11,892	23.90%
Home & Kitchen	₹10,895	21.89%
Beauty	₹6,192	12.44%
Books	₹3,294	6.62%

8.2 Customer performance

Customer	Valid Orders	Spend
Aarav Sharma	3	₹11,192
Aditya Verma	2	₹9,594
Riya Mehta	2	₹9,096
Pooja Malhotra	1	₹4,697
Vikram Rao	1	₹4,297
Karan Singh	1	₹3,097

Ananya Roy	1	₹2,997
Rahul Jain	1	₹2,498
Neha Kapoor	1	₹2,297
Sneha Gupta	0	₹0

8.3 Customer segmentation

Segment	Customers	Revenue	Average Spend
High Value	3	₹29,882	₹9,960.67
Medium Value	3	₹12,091	₹4,030.33
Low Value	4	₹7,792	₹1,948.00

9. Business Insights

- Electronics is the largest revenue category, contributing 35.15% of valid sales revenue.
- Electronics, Clothing and Home & Kitchen together contribute approximately 80.92% of total sales revenue.
- The three High Value customers contribute approximately 60.04% of total sales revenue, showing meaningful customer concentration.
- Aarav Sharma is the highest-spending customer and also has the highest number of valid orders.
- Coffee Maker, Smart Watch, Bluetooth Speaker and Running Shoes all have stock below 50 and recorded valid demand, so they merit inventory monitoring.
- Revenue falls from ₹32,877 in May to ₹16,888 in June, a 48.63% decrease. With only two months of data, I treat this as a prompt for deeper investigation rather than a long-term trend.
- Delhi generates the highest city-level revenue at ₹18,387.

10. Business Recommendations

- Monitor the top three revenue categories closely because they account for most of the current sales base.
- Create retention and engagement analysis for High Value and repeat customers because a small customer group contributes a large share of revenue.
- Introduce inventory alerts for low-stock products with demonstrated demand.
- Investigate the May-to-June revenue decline using order volume, active customers, product mix and order status before taking action.
- Compare city revenue with customer count and average spend so market size is not confused with customer quality.
- In a production environment, add reconciliation checks between calculated order value and payment records, especially for refunds and multi-payment scenarios.

11. Complete SQL Analysis

Below are the 34 business questions and the finalized SQL solutions I used for the project.

Q1. Calculate total revenue from all non-cancelled orders.

```
SELECT SUM(p.price * oi.quantity) AS total_revenue
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.status <> 'Cancelled';
```

Q2. Calculate total revenue by month, excluding cancelled orders.

```
SELECT DATE_FORMAT(o.order_date,'%Y-%m') AS month,
       SUM(p.price * oi.quantity) AS monthly_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY DATE_FORMAT(o.order_date,'%Y-%m')
ORDER BY month;
```

Q3. Calculate average order value for non-cancelled orders.

```
SELECT SUM(p.price * oi.quantity) / COUNT(DISTINCT o.order_id) AS average_order_value
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled';
```

Q4. Calculate revenue by product category.

```
SELECT c.category_name, SUM(p.price * oi.quantity) AS revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN categories c ON p.category_id = c.category_id
WHERE o.status <> 'Cancelled'
GROUP BY c.category_id, c.category_name
ORDER BY revenue DESC;
```

Q5. Calculate revenue by customer city.

```
SELECT c.city, SUM(p.price * oi.quantity) AS revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY c.city
ORDER BY revenue DESC;
```

Q6. Analyze the number and percentage of orders by status.

```
SELECT status, COUNT(*) AS total_orders,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM orders),2) AS percentage_of_orders
FROM orders
GROUP BY status
ORDER BY total_orders DESC;
```

Q7. Analyze paid orders and paid amount by payment method.

```
SELECT payment_method,
       COUNT(*) AS total_orders,
       ROUND(SUM(amount),2) AS total_paid_amount,
       ROUND(SUM(amount) * 100.0 /
             (SELECT SUM(amount) FROM payments WHERE payment_status='Paid'),2)
       AS revenue_contribution_percentage
FROM payments
WHERE payment_status='Paid'
GROUP BY payment_method
ORDER BY total_paid_amount DESC;
```

Q8. Measure quantity sold, revenue and distinct orders for each product.

```
SELECT p.product_id, p.product_name,
       SUM(oi.quantity) AS quantity_sold,
       SUM(p.price * oi.quantity) AS revenue,
       COUNT(DISTINCT o.order_id) AS distinct_orders
FROM products p
JOIN order_items oi ON p.product_id=oi.product_id
JOIN orders o ON oi.order_id=o.order_id
WHERE o.status <> 'Cancelled'
GROUP BY p.product_id,p.product_name
ORDER BY revenue DESC;
```

Q9. Calculate every customer's valid orders and total spend.

```
SELECT c.customer_id,c.customer_name,
       COUNT(DISTINCT o.order_id) AS total_valid_orders,
       COALESCE(SUM(p.price*oi.quantity),0) AS total_spend
FROM customers c
LEFT JOIN orders o ON c.customer_id=o.customer_id AND o.status <> 'Cancelled'
LEFT JOIN order_items oi ON o.order_id=oi.order_id
LEFT JOIN products p ON oi.product_id=p.product_id
GROUP BY c.customer_id,c.customer_name
ORDER BY total_spend DESC;
```

Q10. Identify the top five customers by spending.

```
SELECT c.customer_id,c.customer_name,SUM(p.price*oi.quantity) AS total_spend
FROM customers c
JOIN orders o ON c.customer_id=o.customer_id
JOIN order_items oi ON o.order_id=oi.order_id
JOIN products p ON oi.product_id=p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY c.customer_id,c.customer_name
ORDER BY total_spend DESC
LIMIT 5;
```

Q11. Identify customers with more than one non-cancelled order.

```
SELECT c.customer_id,c.customer_name,COUNT(DISTINCT o.order_id) AS valid_orders
FROM customers c
JOIN orders o ON c.customer_id=o.customer_id
WHERE o.status <> 'Cancelled'
GROUP BY c.customer_id,c.customer_name
HAVING COUNT(DISTINCT o.order_id)>1
ORDER BY valid_orders DESC;
```

Q12. Identify customers who have never placed an order.

```
SELECT c.customer_id,c.customer_name,c.city,c.signup_date
FROM customers c
LEFT JOIN orders o ON c.customer_id=o.customer_id
WHERE o.order_id IS NULL;
```

Q13. Find first order date, latest order date and valid order count for each customer who ordered.

```
SELECT c.customer_id,c.customer_name,
       MIN(o.order_date) AS first_order_date,
       MAX(o.order_date) AS latest_order_date,
       COUNT(DISTINCT o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id=o.customer_id
WHERE o.status <> 'Cancelled'
GROUP BY c.customer_id,c.customer_name
ORDER BY first_order_date;
```

Q14. Identify customers whose spending is above the average spending of customers with valid orders.

```
WITH customer_spending AS (
  SELECT c.customer_id,c.customer_name,SUM(p.price*oi.quantity) AS total_spend
  FROM customers c
  JOIN orders o ON c.customer_id=o.customer_id
  JOIN order_items oi ON o.order_id=oi.order_id
  JOIN products p ON oi.product_id=p.product_id
  WHERE o.status <> 'Cancelled'
  GROUP BY c.customer_id,c.customer_name
)
SELECT customer_id,customer_name,total_spend
FROM customer_spending
WHERE total_spend > (SELECT AVG(total_spend) FROM customer_spending)
ORDER BY total_spend DESC;
```

Q15. Analyze customer count, valid orders, revenue and average revenue per purchasing customer by city.

```
SELECT c.city,
       COUNT(DISTINCT c.customer_id) AS total_customers,
       COUNT(DISTINCT o.order_id) AS total_orders,
       SUM(p.price*oi.quantity) AS total_revenue,
       SUM(p.price*oi.quantity)/COUNT(DISTINCT c.customer_id) AS avg_revenue_per_customer
FROM customers c
JOIN orders o ON c.customer_id=o.customer_id
JOIN order_items oi ON o.order_id=oi.order_id
JOIN products p ON oi.product_id=p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY c.city
ORDER BY total_revenue DESC;
```

Q16. Segment customers into High, Medium and Low Value groups.

```
WITH customer_spending AS (  
  SELECT c.customer_id,c.customer_name,  
         COALESCE(SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END),0) AS total_spend  
  FROM customers c  
  LEFT JOIN orders o ON c.customer_id=o.customer_id  
  LEFT JOIN order_items oi ON o.order_id=oi.order_id  
  LEFT JOIN products p ON oi.product_id=p.product_id  
  GROUP BY c.customer_id,c.customer_name  
)  
SELECT customer_id,customer_name,total_spend,  
       CASE WHEN total_spend >= 7000 THEN 'High Value'  
            WHEN total_spend >= 3000 THEN 'Medium Value'  
            ELSE 'Low Value' END AS customer_segment  
FROM customer_spending  
ORDER BY total_spend DESC;
```

Q17. Identify the top five products by quantity sold.

```
SELECT p.product_id,p.product_name,SUM(oi.quantity) AS quantity_sold  
FROM products p  
JOIN order_items oi ON p.product_id=oi.product_id  
JOIN orders o ON oi.order_id=o.order_id  
WHERE o.status <> 'Cancelled'  
GROUP BY p.product_id,p.product_name  
ORDER BY quantity_sold DESC  
LIMIT 5;
```

Q18. Identify the top five products by revenue.

```
SELECT p.product_id,p.product_name,SUM(p.price*oi.quantity) AS revenue  
FROM products p  
JOIN order_items oi ON p.product_id=oi.product_id  
JOIN orders o ON oi.order_id=o.order_id  
WHERE o.status <> 'Cancelled'  
GROUP BY p.product_id,p.product_name  
ORDER BY revenue DESC  
LIMIT 5;
```

Q19. Identify products that never appear in a non-cancelled order.

```
SELECT p.product_id,p.product_name,c.category_name  
FROM products p  
JOIN categories c ON p.category_id=c.category_id  
LEFT JOIN order_items oi ON p.product_id=oi.product_id  
LEFT JOIN orders o ON oi.order_id=o.order_id AND o.status <> 'Cancelled'  
GROUP BY p.product_id,p.product_name,c.category_name  
HAVING COUNT(o.order_id)=0;
```

Q20. Calculate the average catalogue price per category.

```
SELECT c.category_id,c.category_name,ROUND(AVG(p.price),2) AS avg_product_price  
FROM categories c  
JOIN products p ON c.category_id=p.category_id  
GROUP BY c.category_id,c.category_name  
ORDER BY avg_product_price DESC;
```

Q21. Identify products priced above their own category average.

```
WITH product_prices AS (  
  SELECT p.product_id,p.product_name,p.price,c.category_name,  
         AVG(p.price) OVER (PARTITION BY p.category_id) AS category_avg_price  
  FROM products p  
  JOIN categories c ON p.category_id=c.category_id  
)  
SELECT product_id,product_name,category_name,price,ROUND(category_avg_price,2) AS category_avg_price  
FROM product_prices  
WHERE price > category_avg_price  
ORDER BY category_name,price DESC;
```

Q22. Identify products with stock below 50 that have generated valid sales.

```
SELECT p.product_id,p.product_name,p.stock,  
       SUM(CASE WHEN o.status <> 'Cancelled' THEN oi.quantity ELSE 0 END) AS quantity_sold,  
       SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END) AS revenue  
FROM products p  
LEFT JOIN order_items oi ON p.product_id=oi.product_id  
LEFT JOIN orders o ON oi.order_id=o.order_id  
GROUP BY p.product_id,p.product_name,p.stock  
HAVING p.stock < 50 AND quantity_sold > 0  
ORDER BY quantity_sold DESC;
```

Q23. Analyze product count, quantity sold, revenue and average catalogue price by category.

```
WITH category_catalogue AS (  
  SELECT c.category_id,c.category_name,  
         COUNT(p.product_id) AS total_products,  
         ROUND(AVG(p.price),2) AS average_product_price  
  FROM categories c  
  LEFT JOIN products p ON c.category_id=p.category_id  
  GROUP BY c.category_id,c.category_name  
)  
,  
category_sales AS (  
  SELECT p.category_id,  
         SUM(oi.quantity) AS quantity_sold,  
         SUM(p.price*oi.quantity) AS revenue  
  FROM products p  
  JOIN order_items oi ON p.product_id=oi.product_id  
  JOIN orders o ON oi.order_id=o.order_id  
  WHERE o.status <> 'Cancelled'  
  GROUP BY p.category_id  
)  
SELECT cc.category_id,cc.category_name,cc.total_products,  
       COALESCE(cs.quantity_sold,0) AS quantity_sold,  
       COALESCE(cs.revenue,0) AS revenue,  
       cc.average_product_price  
FROM category_catalogue cc  
LEFT JOIN category_sales cs ON cc.category_id=cs.category_id  
ORDER BY revenue DESC;
```

Q24. Rank products by revenue within each category.

```
WITH product_revenue AS (  
  SELECT p.product_id,p.product_name,c.category_name,  
         COALESCE(SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END),0) AS revenue  
  FROM products p  
  JOIN categories c ON p.category_id=c.category_id  
  JOIN orders o ON p.product_id=o.product_id  
  WHERE o.status <> 'Cancelled'  
  GROUP BY p.product_id,p.product_name,c.category_name  
)  
SELECT product_id,product_name,category_name,revenue  
FROM product_revenue  
ORDER BY category_name,product_id, revenue DESC;
```

```

LEFT JOIN order_items oi ON p.product_id=oi.product_id
LEFT JOIN orders o ON oi.order_id=o.order_id
GROUP BY p.product_id,p.product_name,c.category_name
)
SELECT product_id,product_name,category_name,ROUND(revenue,2) AS revenue,
       RANK() OVER (PARTITION BY category_name ORDER BY revenue DESC) AS category_rank
FROM product_revenue
ORDER BY category_name,category_rank;

```

Q25. Return the top three products per category by revenue.

```

WITH product_revenue AS (
  SELECT p.product_id,p.product_name,c.category_name,
         COALESCE(SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END),0) AS revenue
  FROM products p
  JOIN categories c ON p.category_id=c.category_id
  LEFT JOIN order_items oi ON p.product_id=oi.product_id
  LEFT JOIN orders o ON oi.order_id=o.order_id
  GROUP BY p.product_id,p.product_name,c.category_name
), ranked_products AS (
  SELECT *,ROW_NUMBER() OVER (PARTITION BY category_name ORDER BY revenue DESC) AS product_rank
  FROM product_revenue
)
SELECT product_id,product_name,category_name,ROUND(revenue,2) AS revenue,product_rank
FROM ranked_products
WHERE product_rank <= 3
ORDER BY category_name,product_rank;

```

Q26. Rank customers by spending using DENSE_RANK().

```

WITH customer_spending AS (
  SELECT c.customer_id,c.customer_name,
         COALESCE(SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END),0) AS total_spend
  FROM customers c
  LEFT JOIN orders o ON c.customer_id=o.customer_id
  LEFT JOIN order_items oi ON o.order_id=oi.order_id
  LEFT JOIN products p ON oi.product_id=p.product_id
  GROUP BY c.customer_id,c.customer_name
)
SELECT customer_id,customer_name,ROUND(total_spend,2) AS total_spend,
       DENSE_RANK() OVER (ORDER BY total_spend DESC) AS spending_rank
FROM customer_spending
ORDER BY spending_rank;

```

Q27. Return the top two customers by spending in each city.

```

WITH customer_spending AS (
  SELECT c.customer_id,c.customer_name,c.city,
         COALESCE(SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END),0) AS total_spend
  FROM customers c
  LEFT JOIN orders o ON c.customer_id=o.customer_id
  LEFT JOIN order_items oi ON o.order_id=oi.order_id
  LEFT JOIN products p ON oi.product_id=p.product_id
  GROUP BY c.customer_id,c.customer_name,c.city
), ranked_customers AS (
  SELECT *,ROW_NUMBER() OVER (PARTITION BY city ORDER BY total_spend DESC) AS city_rank
  FROM customer_spending
)
SELECT customer_id,customer_name,city,ROUND(total_spend,2) AS total_spend,city_rank
FROM ranked_customers

```

```
WHERE city_rank <= 2
ORDER BY city,city_rank;
```

Q28. Calculate monthly revenue and cumulative running revenue.

```
WITH monthly_revenue AS (
  SELECT DATE_FORMAT(o.order_date,'%Y-%m') AS month,SUM(p.price*oi.quantity) AS revenue
  FROM orders o
  JOIN order_items oi ON o.order_id=oi.order_id
  JOIN products p ON oi.product_id=p.product_id
  WHERE o.status <> 'Cancelled'
  GROUP BY DATE_FORMAT(o.order_date,'%Y-%m')
)
SELECT month,ROUND(revenue,2) AS monthly_revenue,
       ROUND(SUM(revenue) OVER (ORDER BY month),2) AS cumulative_revenue
FROM monthly_revenue
ORDER BY month;
```

Q29. Calculate each category's percentage contribution to total revenue.

```
WITH category_revenue AS (
  SELECT c.category_id,c.category_name,SUM(p.price*oi.quantity) AS revenue
  FROM categories c
  JOIN products p ON c.category_id=p.category_id
  JOIN order_items oi ON p.product_id=oi.product_id
  JOIN orders o ON oi.order_id=o.order_id
  WHERE o.status <> 'Cancelled'
  GROUP BY c.category_id,c.category_name
)
SELECT category_name,ROUND(revenue,2) AS revenue,
       ROUND(revenue*100.0/SUM(revenue) OVER (),2) AS revenue_contribution_percentage
FROM category_revenue
ORDER BY revenue DESC;
```

Q30. Calculate month-over-month revenue growth using LAG().

```
WITH monthly_revenue AS (
  SELECT DATE_FORMAT(o.order_date,'%Y-%m') AS month,SUM(p.price*oi.quantity) AS revenue
  FROM orders o
  JOIN order_items oi ON o.order_id=oi.order_id
  JOIN products p ON oi.product_id=p.product_id
  WHERE o.status <> 'Cancelled'
  GROUP BY DATE_FORMAT(o.order_date,'%Y-%m')
), monthly_comparison AS (
  SELECT month,revenue,LAG(revenue) OVER (ORDER BY month) AS previous_month_revenue
  FROM monthly_revenue
)
SELECT month,ROUND(revenue,2) AS revenue,ROUND(previous_month_revenue,2) AS previous_month_revenue,
       CASE WHEN previous_month_revenue IS NULL OR previous_month_revenue=0 THEN NULL
       ELSE ROUND((revenue-previous_month_revenue)*100.0/previous_month_revenue,2) END AS mom_growth_percentage
FROM monthly_comparison
ORDER BY month;
```

Q31. Summarize customer count, revenue and average spend by customer segment.

```
WITH customer_spending AS (
  SELECT c.customer_id,c.customer_name,
         COALESCE(SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END),0) AS total_spend
  FROM customers c
```

```

LEFT JOIN orders o ON c.customer_id=o.customer_id
LEFT JOIN order_items oi ON o.order_id=oi.order_id
LEFT JOIN products p ON oi.product_id=p.product_id
GROUP BY c.customer_id,c.customer_name
), segmented_customers AS (
  SELECT customer_id,customer_name,total_spend,
    CASE WHEN total_spend >= 7000 THEN 'High Value'
      WHEN total_spend >= 3000 THEN 'Medium Value'
      ELSE 'Low Value' END AS customer_segment
  FROM customer_spending
)
SELECT customer_segment,COUNT(*) AS customer_count,
  ROUND(SUM(total_spend),2) AS total_revenue,
  ROUND(AVG(total_spend),2) AS avg_revenue_per_customer
FROM segmented_customers
GROUP BY customer_segment
ORDER BY total_revenue DESC;

```

Q32. Compare no-order, one-time and repeat customers.

```

WITH customer_orders AS (
  SELECT c.customer_id,c.customer_name,
    COUNT(DISTINCT CASE WHEN o.status <> 'Cancelled' THEN o.order_id END) AS valid_orders,
    COALESCE(SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END),0) AS total_spend
  FROM customers c
  LEFT JOIN orders o ON c.customer_id=o.customer_id
  LEFT JOIN order_items oi ON o.order_id=oi.order_id
  LEFT JOIN products p ON oi.product_id=p.product_id
  GROUP BY c.customer_id,c.customer_name
), customer_type AS (
  SELECT customer_id,customer_name,valid_orders,total_spend,
    CASE WHEN valid_orders=0 THEN 'No Orders'
      WHEN valid_orders=1 THEN 'One-Time Customer'
      ELSE 'Repeat Customer' END AS customer_type
  FROM customer_orders
)
SELECT customer_type,COUNT(*) AS customer_count,
  ROUND(SUM(total_spend),2) AS total_revenue,
  ROUND(AVG(total_spend),2) AS avg_spending
FROM customer_type
GROUP BY customer_type
ORDER BY customer_count DESC;

```

Q33. Classify the product portfolio into revenue bands.

```

WITH product_sales AS (
  SELECT p.product_id,p.product_name,
    COALESCE(SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END),0) AS revenue,
    COALESCE(SUM(CASE WHEN o.status <> 'Cancelled' THEN oi.quantity ELSE 0 END),0) AS quantity_sold
  FROM products p
  LEFT JOIN order_items oi ON p.product_id=oi.product_id
  LEFT JOIN orders o ON oi.order_id=o.order_id
  GROUP BY p.product_id,p.product_name
), classified_products AS (
  SELECT product_id,product_name,revenue,quantity_sold,
    CASE WHEN revenue=0 THEN 'No Sales'
      WHEN revenue >= 5000 THEN 'High Revenue'
      WHEN revenue >= 2000 THEN 'Medium Revenue'
      ELSE 'Low Revenue' END AS revenue_category
  FROM product_sales
)

```

```

SELECT revenue_category,COUNT(*) AS product_count,
       ROUND(SUM(revenue),2) AS total_revenue,SUM(quantity_sold) AS total_quantity_sold
FROM classified_products
GROUP BY revenue_category
ORDER BY total_revenue DESC;

```

Q34. Create a one-row executive KPI summary using CTEs.

```

WITH customer_summary AS (
  SELECT COUNT(*) AS total_customers FROM customers
), order_summary AS (
  SELECT COUNT(*) AS total_orders,
         SUM(CASE WHEN status <> 'Cancelled' THEN 1 ELSE 0 END) AS valid_orders,
         SUM(CASE WHEN status='Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders
  FROM orders
), revenue_summary AS (
  SELECT SUM(CASE WHEN o.status <> 'Cancelled' THEN p.price*oi.quantity ELSE 0 END) AS total_revenue,
         COUNT(DISTINCT CASE WHEN o.status <> 'Cancelled' THEN o.order_id END) AS valid_order_count,
         SUM(CASE WHEN o.status <> 'Cancelled' THEN oi.quantity ELSE 0 END) AS total_quantity_sold
  FROM orders o
  JOIN order_items oi ON o.order_id=oi.order_id
  JOIN products p ON oi.product_id=p.product_id
), product_summary AS (
  SELECT COUNT(*) AS total_products FROM products
)
SELECT cs.total_customers,os.total_orders,os.valid_orders,os.cancelled_orders,
       ROUND(rs.total_revenue,2) AS total_revenue,
       ROUND(rs.total_revenue/rs.valid_order_count,2) AS average_order_value,
       ps.total_products,rs.total_quantity_sold
FROM customer_summary cs
CROSS JOIN order_summary os
CROSS JOIN revenue_summary rs
CROSS JOIN product_summary ps;

```

12. Technical Validation Notes

- Q12 intentionally answers “customers who have never placed an order.” In this dataset no customer qualifies because Sneha Gupta has a cancelled order; if the business question were “no valid orders,” the status filter would be moved into the LEFT JOIN.
- Q21 calculates the category average before filtering, so the comparison is against the complete category catalogue.
- Q23 separates catalogue metrics from transaction metrics so repeated order-item rows do not distort the average product price.
- Q32 uses COUNT(DISTINCT order_id) to ensure an order with multiple items is counted once.
- The payments table contains manually supplied amounts. Therefore, I treat sales revenue (price × quantity) and paid payment amount as two different analytical measures rather than assuming they reconcile exactly.

13. Dashboard Specification

Dashboard Element	Recommended Visual	Purpose
Total Revenue	KPI card	Overall valid sales revenue
Valid Orders	KPI card	Transaction volume

AOV	KPI card	Value per valid order
Cancellation Rate	KPI card	Order cancellation share
Monthly Revenue	Column/line chart	Short-period revenue movement
Revenue by Category	Horizontal bar	Category concentration
Top Products	Horizontal bar	Leading products by revenue
Customer Segments	Bar/donut	Customer-value concentration
Revenue by City	Bar chart	Geographic comparison
Payment Method	Bar/donut	Paid amount by method
Low-Stock Products	Table	Inventory attention list

14. GitHub / Portfolio Structure

```
ecommerce-sql-project/
├── README.md
├── sql/
│   ├── 01_database_setup.sql
│   ├── 02_business_questions.sql
│   └── 03_final_queries.sql
├── diagrams/
│   └── ecommerce_er_diagram.png
├── documentation/
│   └── Ecommerce_SQL_Project_Final_Portfolio_Documentation.docx
└── dashboard/
    └── ecommerce_dashboard.pbix
```

15. How I Would Explain This Project in an Interview

I built a six-table e-commerce database in MySQL and used SQL to analyze revenue, customer behavior, product performance, inventory, geography and payments. I started by defining the business questions, then used joins and aggregations for core reporting, CTEs and subqueries for multi-step logic, and window functions such as RANK, DENSE_RANK, ROW_NUMBER, LAG and cumulative SUM for advanced analysis. I validated the outputs against the dataset and translated the findings into business insights and dashboard requirements. The project is intentionally compact, but the analytical patterns are directly transferable to larger datasets.

16. Project Limitations & Future Enhancements

- The dataset is intentionally small and covers only two months.
- The schema does not include discounts, product cost, taxes, shipping, returns or supplier lead times.
- Profitability cannot be calculated without product cost.
- Long-term retention and cohort analysis require a longer transaction history.
- A production version should include data-quality checks and order/payment reconciliation.
- The next extension is an executive dashboard in Power BI or Excel using the verified SQL outputs.

17. Final Project Checklist

✓ Database created

- ✓ Six tables and relationships completed
- ✓ Dataset loaded
- ✓ ER structure documented
- ✓ 34 business questions completed
- ✓ SQL solutions audited
- ✓ Executive KPIs verified
- ✓ Business insights written
- ✓ Recommendations written
- ✓ Dashboard specification prepared
- ✓ Interview explanation prepared
- ✓ GitHub structure defined

18. Conclusion

I created this project as a complete SQL analytics case study rather than a collection of isolated practice queries. It demonstrates my ability to understand a business problem, design and navigate a relational database, write analytical SQL, validate results, interpret business performance and communicate findings clearly. The project is ready to be used as the documentation component of my portfolio, with the dashboard as the next enhancement.